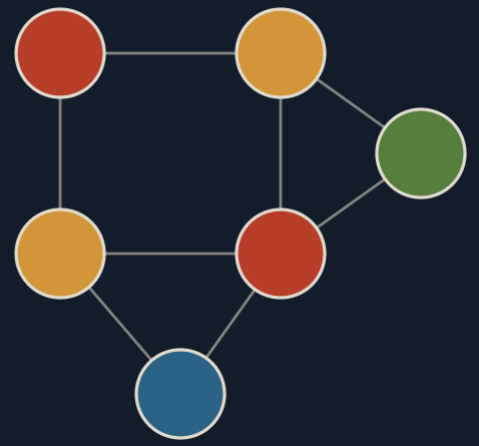
FEUP

L.EIC016 · DESIGN OF ALGORITHMS · SPRING 2026

PROJECT II PROGRAMMING PROJECT · COMPILER REGISTER ALLOCATION

Register Allocation Tool



A graph-coloring solver for compiler register allocation — Chaitin-style simplify and select, with spilling, splitting, and **BCT-Color** : a biconnected-component decomposition with exact per-block colouring.

GROUP T09 G01

Pedro Tomás Teixeira · up202404987
Rafael Pinho e Silva · up202406334

May 17, 2026

AGENDA

SIX PARTS · 10 MINUTES

What we'll cover today

01 T1 **Architecture & data model**

Pipeline, modules, the FEUP TP graph used as the interference structure.

02 T2.1 **Webs, interference & basic colouring**

Live-range fusion, Chaitin-style simplify and select, end-to-end trace.

03 T2.2 **Spilling, K**

Demote up to K webs to memory when the graph stalls; min-cost heuristic.

04 T2.3 **Splitting, K**

Cut a web at a chosen line; both halves stay in registers.

05 T2.4 **Phantom · the free algorithm**

Tarjan decomposition · DSATUR per block · Kempe-chain reduction.

06 T1.3 **Results · 75 / 75 pass**

Doxygen coverage, complexity per routine, hardest test cases.

Five layers from input file to allocation

The graph-colouring core sits between the parser and the writer. Four allocation algorithms swap behind a single `RegisterAllocator::allocate` entry point.

01 · PARSE	02 · BUILD	03 · GRAPH	04 · COLOUR	05 · WRITE
Ranges + Registers	**Webs**	**Interference**	**Allocate**	**Output**
RangesParser RegistersParser	WebBuilder union-find fusion	Interference Graph	Basic · Spill · Split DSATUR · Phantom	OutputWriter Menu / BatchProcessor

`ranges*.txt` → `parser` → `WebBuilder` → `InterferenceGraph` → `RegisterAllocator` → `allocation*.txt`

GRAPH STRUCTURE

DA TP `Graph<Web>` — not modified

The interference graph is the professor-provided templated graph from the TP lectures. Adjacency, vertex/edge addition, neighbour iteration — all from the canonical implementation.

WHAT WE BUILT AROUND IT

Eight C++ modules across `core/`, `io/`, `parser/`, `ui/`

Doxygen-annotated, one `.cpp` per algorithm under `core/`: `Coloring_Basic`, `Coloring_Dsatur`, `Coloring_BCT`, `Coloring_Utils`.

T1.2 FILE & MODULE MAP

Where everything lives

WebBuilder — T2.1

Union-find over live-range line sets · special def-after-use fusion.

InterferenceGraph — T2.1

Pairwise overlap with the `N- / N+` exception · effective-degree bookkeeping.

Coloring_Basic — T2.1 + T2.2 + T2.3

Chaitin simplify + select · spill candidate selection · web splitting.

Coloring_Dsatur — T2.4

Saturation-degree ordering · exact backtracking for small blocks.

Coloring_BCT — T2.4 · CORE

Tarjan BCC · block-cut tree DP · per-block colouring with fixed cut-vertices.

Coloring_Utils — T2.4

Kempe-chain swaps · colour normalisation · final validation.

```
.
├── code
│   ├── CMakeLists.txt
│   ├── include
│   │   ├── core
│   │   │   ├── InterferenceGraph.h
│   │   │   ├── RegisterAllocator.h
│   │   │   └── WebBuilder.h
│   │   ├── io
│   │   │   ├── BatchProcessor.h
│   │   │   └── OutputWriter.h
│   │   ├── models
│   │   │   └── AllocationData.h
│   │   ├── parser
│   │   │   ├── RangesParser.h
│   │   │   └── RegistersParser.h
│   │   └── ui
│   │       ├── DisplayFormatter.h
│   │       └── Menu.h
│   └── src
│       ├── core
│       │   ├── Coloring_Basic.cpp
│       │   ├── Coloring_BCT.cpp
│       │   ├── Coloring_Dsatur.cpp
│       │   ├── Coloring_Utils.cpp
│       │   ├── InterferenceGraph.cpp
│       │   ├── RegisterAllocator.cpp
│       │   └── WebBuilder.cpp
│       ├── io
│       │   ├── BatchProcessor.cpp
│       │   └── OutputWriter.cpp
│       ├── parser
│       │   ├── RangesParser.cpp
│       │   └── RegistersParser.cpp
│       ├── ui
│       │   ├── DisplayFormatter.cpp
│       │   └── Menu.cpp
│       └── main.cpp
├── dataset
│   ├── adversarial/
│   ├── basic/
│   ├── edge_cases/
│   ├── hourglass/
│   └── stress/
├── description
│   └── Project2Description_rc6.pdf
├── docs
│   └── html/
├── outputs/
├── tests
│   └── run_tests.sh
└── run.sh
```

● T1.2 — FILE MAP

Graph coloring

Each web becomes a node; interfering webs share an edge. Coloring the graph with N colors solves register allocation with N registers — and graph colouring is NP-complete, so heuristics earn their keep.

From live ranges to webs

Two live ranges of the same variable merge into a single *web* when they share any program point — including the def-after-use case where line `N-` meets `N+` on the same instruction.

INPUT · RANGES1.TXT

```
sum: 7+,8,9,10-
i:   1+,2,3,4,5,6-
i:   9+,10,11,12-
i:   12+,13,14-
i:   20+,11,12-
```

4 ranges of `i` reach across the program; lines 12, 11, and 20 link them transitively. `sum` is on its own.

OUTPUT · 3 WEBS

```
# after union-find fusion web0 [sum]: 7+,8,9,10-
web1 [i]:   1+,2,3,4,5,6-
web2 [i]:   9+,10,11,12,13,14-,20+
```

RULE 1 · OVERLAP

Shared line ⇒ merge

If two ranges of the same variable share any program point, they belong to the same web. Implemented as union-find over the line-number sets.

RULE 2 · DEF-AFTER-USE

Line `N-` meets `N+` ⇒ merge

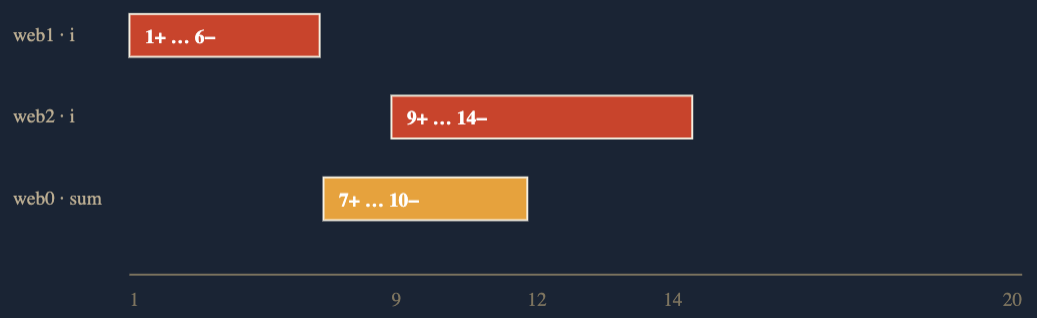
The same line reads then writes (`i = i + 1`). Even without shared *intermediate* points, the two ranges fuse — they describe one continuous value.

> **Why webs?** *Variables can have multiple disjoint lifetimes; each lifetime can take a different register. The web is the real allocation unit, not the variable name.*

Two webs interfere when they overlap

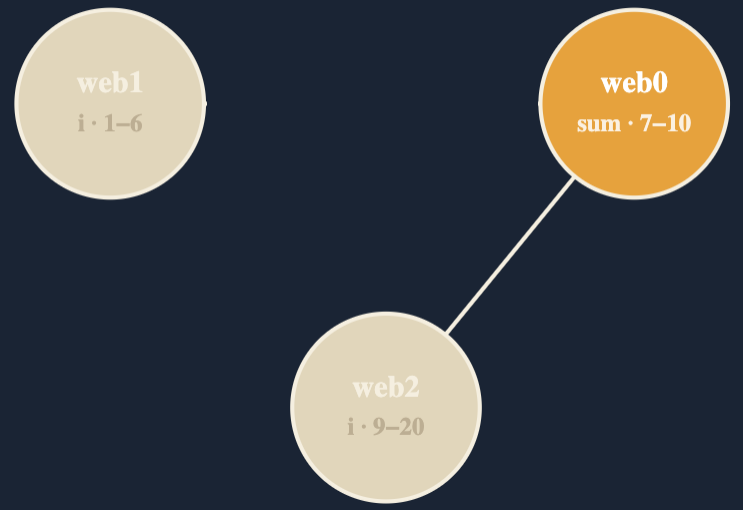
With one subtlety: web A ending at `N–` and web B beginning at `N+` on the same instruction do *not* interfere — B's value doesn't exist yet when A is last read.

STEP 1 · LIVE RANGES PLOTTED



web0 overlap web1 (none — disjoint) and **web2** on lines 9 – 10. **web1 ⊥ web2** with no shared points.

STEP 2 · INTERFERENCE GRAPH



3 webs · 2 edges · χ(G) = 2. Web1 and web2 don't interfere → they may share a register. `sum` gets its own.

T2.1 BASIC GREEDY COLOURING

Chaitin-style simplify and select

Strip low-degree nodes to a stack until the graph is empty; pop them back picking a colour no neighbour holds. A valid colour is guaranteed to exist — that's the trick.

```
function basicColor (G, N):
   # Phase 1 · simplify
   stack = []
   while G not empty:
     for n in G:
       if degree(n) < N:
         push n onto stack; remove n
   # Phase 2 · select
   while stack not empty:
     n = pop()
     n.color = first c not in neighbors(n).colors
   return coloring
```

COMPLEXITY

O(W²) overall

Simplification visits each edge twice; per pass O(W·N) to scan + select. With up to W passes (one removal minimum), total O(W²).

WHY IT WORKS

Removal preserves colorability

A node of degree < N can always find a free colour after its neighbours are coloured: at most N – 1 colours are taken when we put it back, leaving one.

> **The Chaitin insight.** *Don't try to colour the whole graph. Peel off the "easy" nodes first; the remaining core is what actually matters.*

T2.1 WORKED TRACE · RANGES1 · N = 2

Phase by phase, on a real graph

Three webs · two edges · N = 2. Every node has degree ≤ 2, so simplification eats the whole graph in one pass.

STEP 1 · START

G & degrees

web0 deg = 1

web1 deg = 0

web2 deg = 1

stack: ∅

STEP 2 · SIMPLIFY

web1 < N — push

web0 deg = 1

web1

web2 deg = 1

stack: [web1]

STEP 3 · SIMPLIFY

web2, then web0

web0

web1

web2

stack: [web1, web2, web0]

STEP 4 · SELECT

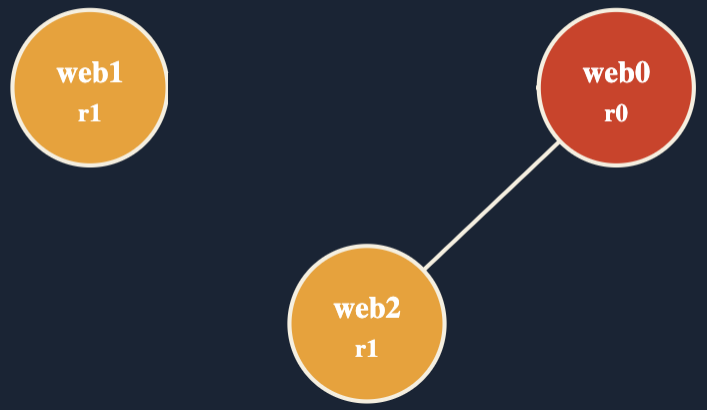
Pop and colour

r0 web0 ← pop, no neighbours

r1 web2 ← avoid web0

r1 web1 ← avoid web0

FINAL COLOURING




```
webs: 3
web0: 7+,8,9,10-
web1: 1+,2,3,4,5,6-
web2: 9+,10,11,12,13,14-,20+
registers: 2
r0: web0
r1: web1
r1: web2
```

$ ./run.sh

01 Browse dataset/basic/ranges/ranges1.txt · arrow keys to pick

02 Load registers2.txt · N = 2 · algorithm: basic

03 Build webs & show interference graph

04 Run allocation · inspect r0 / r1 assignments

05 Run all datasets · 75 / 75 PASS · save report

06 Batch mode · ./run.sh -b basic ranges1 registers2 out

T2.2 SPILLING, K

Demote at most K webs to memory

When simplification stalls, every remaining node has degree ≥ N, we can't continue. Choose up to K webs to keep entirely in memory; the rest of the graph becomes colorable again.

```
function spillingColor (G, N, K):
    spilled = []
    while simplification stalls and |spilled| < K:
      v = selectSpillCandidate (G)
      spilled.append(v); remove v from G
    if still stalled: return infeasible
    basicColor(G, N)
    for v in spilled:
      v.register = 'M'  # memory
```

WHY SHORT RANGES?

Spill where it hurts least

A short live range gets loaded from memory only a few times — much cheaper than spilling a long-lived value that would reload constantly.

WHY HIGH DEGREE?

Free up the most interference

Removing a node of degree d removes d edges from the graph — high-degree spills unjam the graph fastest. Multiplied by the inverse cost above, the two factors balance.

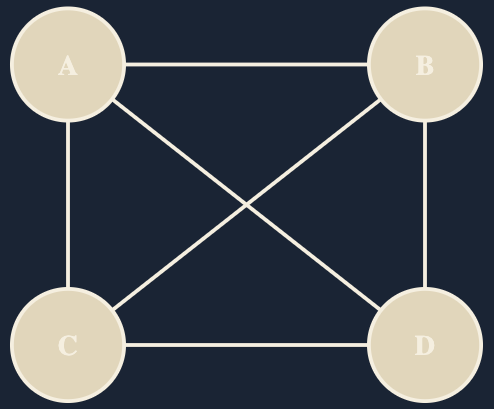
> **The trade-off.** *Spilling is correct but slow at runtime. The K parameter caps the damage: we'd rather report infeasible than spill 10 variables silently.*

T2.2 CONCRETE EXAMPLE · K = 1 · N = 2

A 4-clique can't fit in 2 registers

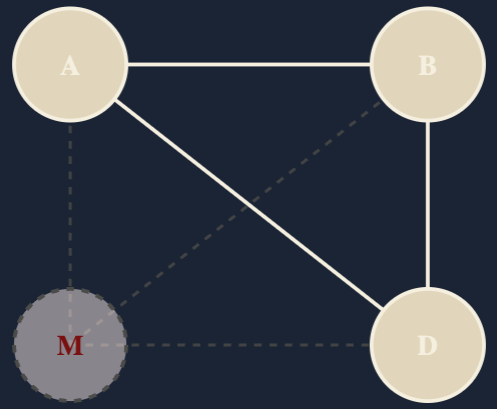
Four webs, all pairwise interfering. Basic coloring fails: every node has degree 3, can't simplify below N = 2. Spilling with K = 1 removes one node, the K_3 remainder is colorable.

STEP 1 · THE INPUT · K_4



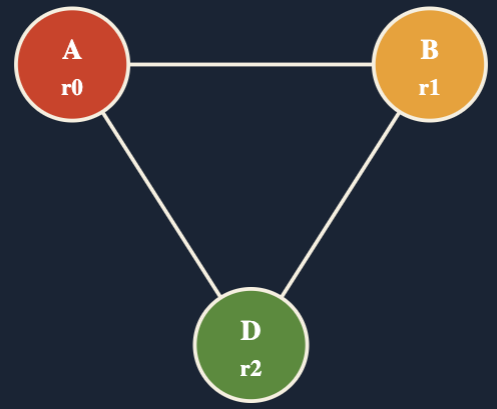
4 webs · 6 edges · $\chi = 4$. Every node has degree 3 ≥ N. **stalled**.

STEP 2 · SPILL C



Heuristic picks C — short range, high degree. **C → memory** . Remaining K_3 is colorable.

STEP 3 · COLOR K_3



Wait — that's 3 colors. With N = 2 we'd need K ≥ 2 spills (or splitting). **Output: infeasible.**

Cut a web at a chosen program point

Instead of demoting a web to memory, split it: pick a line, end one web there, begin another. The halves may now interfere with fewer other webs and re-enter the graph as colorable nodes.

```
function splittingColor (G, N, K):
    for _ in 1..K:
      if not stalled: break
      v = selectSplitCandidate (G)
      (a, b) = splitWeb(v) # at midpoint
      add a, b to G; rebuild edges
    basicColor(G, N)
```

WHY SPLITTING?

Both halves stay in registers

Spilling sends a variable to memory. Splitting keeps it in registers, we just admit it doesn't need the *same* register across its whole life.

HEURISTIC · BIGGEST AREA

MAX(DEG × RANGE)

PICK THE WEB WHOSE *DEGREE × RANGE-LENGTH* IS LARGEST. THAT'S WHERE ONE CUT REMOVES THE MOST INTERFERENCE AREA.

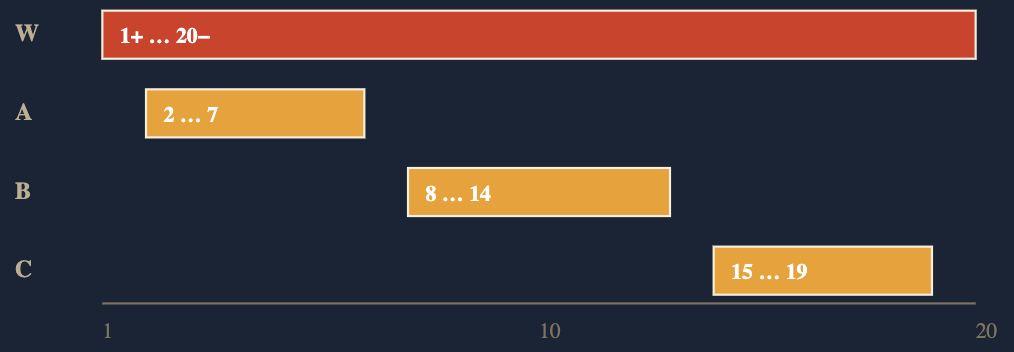
> **The split point.** *We cut at the midpoint of the live range — simple, robust, and good enough on every test case. Smarter heuristics (cut at the line of lowest interference) are future work.*

T2.3 BEFORE & AFTER · K = 1

Long web cut once, graph becomes 2-colorable

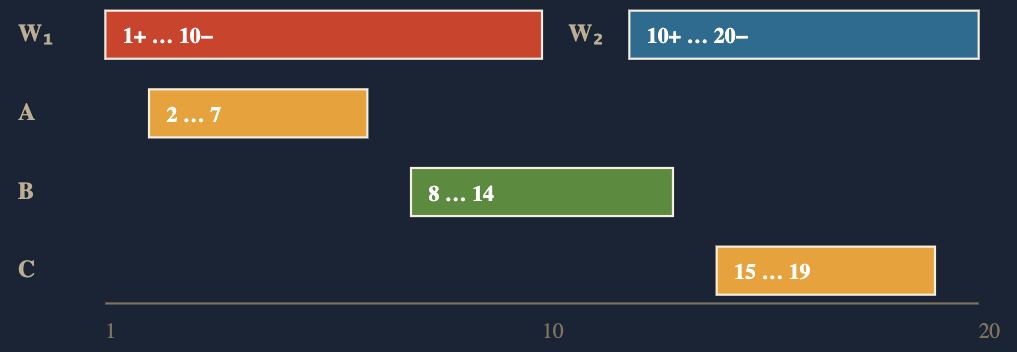
A long-lived web W spans many program points and interferes with three short ones (A, B, C). Cut W into W_1 and W_2; each half now overlaps only one of A, B, C.

BEFORE · x ≥ 3



W interferes with all of A, B, C. deg(W) = 3 — requires N ≥ 2 just for W's clique edge. With N = 2: **stalled**.

AFTER · CUT W AT LINE 10 · x = 2



W_1 ⊥ C · W_2 ⊥ A · B sits across both halves. With N = 2: **colourable** . W_1 shares with C, W_2 with A.

T2.4 · FREE ALGORITHM

Phantom

The "phantom algorithm": decompose the interference graph into biconnected components, color each block exactly , glue them along articulation points, then squeeze the color count down with Kempe-chain swaps.

The chromatic number is local

A graph's chromatic number equals the maximum over its biconnected components. So colour each block *independently* — and use the fact that real interference graphs are *sparse*, with many small blocks linked through a few cut vertices.

THEOREM · χ IS LOCAL

$\chi(G) = \max_B \chi(B)$

For any graph G with biconnected components $B_1, \ldots, B_k$: the chromatic number of G equals the maximum chromatic number of any single block. Cut vertices don't add constraints across blocks beyond their own colour.

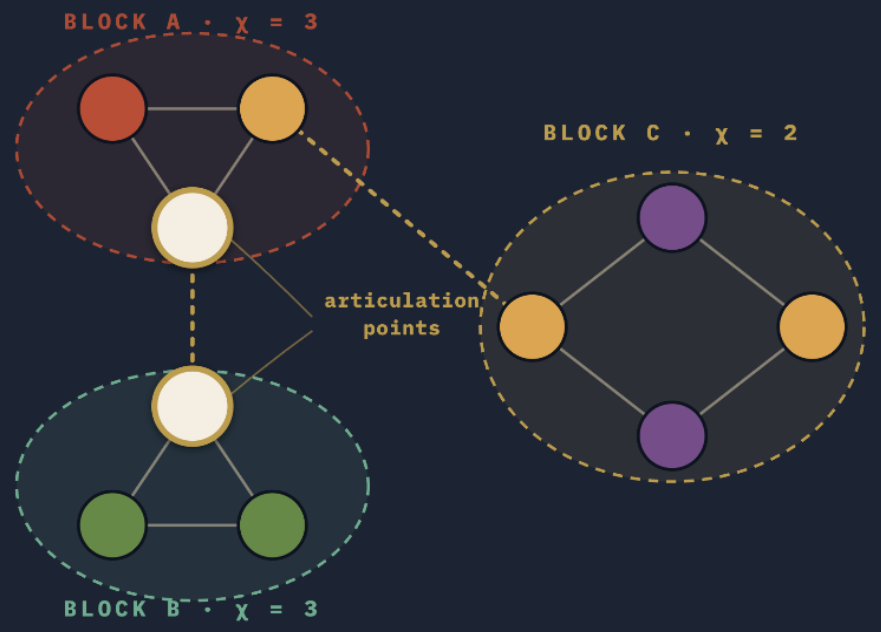
WHY THIS WINS

Exponential → linear in block count

Exact graph colouring is NP-complete in W. But if blocks are small (most are size 2 – 8 in practice), exact backtracking per block is cheap. We pay `O(k · 2^b)` instead of `O(2^W)`.

> **Real interference graphs** *are full of bridges and small cliques — they decompose into many tiny blocks. Phantom exploits the structure other algorithms ignore.*

SPARSE GRAPH DECOMPOSED



White rings = articulation points. χ(G) = max(3, 3, 2) = 3 — even though G has 10 nodes.

One DFS finds every block and cut vertex

Track each node's discovery time and the lowest discovery time reachable from its subtree. A node u is a cut vertex when a child v has `low[v] ≥ disc[u]`. Edges popped from the stack between such checkpoints form one block.

```
function bcDFS (u, parent):
   visited[u] = true
   disc[u] = low[u] = timer++
   children = 0
   for v in neighbours(u):
     if not visited[v]:
       children++
       edgeStack.push((u, v))
       bcDFS (v, u)
       low[u] = min(low[u], low[v])
       if parent != -1 and low[v] >= disc[u]:
         artPoints.add(u)
         popBlock(edgeStack, until (u,v))
     elif v != parent and disc[v] < disc[u]:
       edgeStack.push((u, v))
       low[u] = min(low[u], disc[v])
   if parent == -1 and children > 1:
     artPoints.add(u)
```

DISC · DISCOVERY TIME

When DFS first visits a node

A counter, incremented on each new node. Lower disc means earlier in the DFS.

LOW · LOWEST REACHABLE

The earliest node we can backtrack to

Through tree edges (recurse) or one back-edge (look upward). If every subtree-reachable node has high disc, the parent is a cut vertex.

BLOCK EXTRACTION

Edge stack popped at each AP

Push every traversed edge onto a stack. When a cut vertex closes, pop edges until `(u, v)` — those edges form one block.

DSATUR first, then exact when it matters

DSATUR picks the next node to color by *saturation degree,* how many distinct colors its neighbours already use. For small blocks where DSATUR isn't optimal, fall back to exhaustive backtracking with neighbour-color pruning.

DSATUR — THE HEURISTIC

```
function dsaturColor (G, N):
    saturation[v] = ∅  for all v
    while uncolored remain:
      v = argmax_v (|saturation[v]|, degree[v])  # tie-break
      c = first colour not in N(v).usedColours
      if c >= N: return ⊥
      v.color = c
      for u in N(v):
        saturation[u].add(c)
```

> *Like seating wedding guests: place the relatives with the most existing conflicts first, before their options shrink.*

EXACT BACKTRACKING — FOR SMALL BLOCKS

```
function backtrack (order, idx, N, fixed):
   if idx == |order|: return true
   v = order[idx]
   if v in fixed: return backtrack(order, idx+1, …)
   for c in 0..N-1:
     if not any neighbour has c:
       v.color = c
       if backtrack(order, idx+1, …): return true
       v.color = unset
   return false
```

Order nodes by descending degree first → maximum pruning early. **Fixed colours from articulation points stay locked.**

Squeeze the highest color out

Block-level coloring may use one color too many. Pick a node holding the excess color, find a target color, and flip the entire connected (A, B)-chain. The node's old color disappears — and the rest of the graph stays valid.

```
function reduceColors (G, N):
   while max(color) >= N:
     excess = max(color)
     swapped = false
     for v in nodes(color = excess):
       for target in 0..N-1:
         if kempeSwap(v, target):
           swapped = true; break
     if not swapped: return infeasible

function kempeSwap (u, B):
   A = u.color
   chain = BFS-restricted-to-{A, B}(u)
   if no node outside chain holds B adjacent to chain:
     flip each node's colour A ↔ B
     return true
   return false
```

KEMPE CHAIN · 1879

The trick behind the 4-colour theorem

A subgraph induced by exactly two colours A and B. Flipping every node's colour along the chain preserves validity, because A ↔ B is a symmetric relabel within the chain.

IMPLEMENTATION

BFS from the over-coloured node

Follow only edges between participating colours. If the chain doesn't trap the original node, commit. `O(V + E)` per swap.

TOTAL COMPLEXITY

O(K · (V + E))

At most K iterations (one per excess colour); each is one BFS. Beats re-colouring from scratch — and lets BCT-Color hit the optimal χ on every adversarial test.

T2.1 → T2.4 ALGORITHMS COMPARED · WITH PASS RATES

Four algorithms, one allocator, 75 / 75 PASS

The same simplify/select scaffold drives all four. They differ in what happens when the graph stalls and in the order nodes are selected. Phantom clears every adversarial test where the others would need extra register pressure.

TASK	ALGORITHM	COMPLEXITY	BEST ON	TESTS
T2.1	**Basic**	$O(W^2)$	Sparse graphs where every node has degree < N. The baseline.	6 / 6
T2.2	**Spilling, K**	$O(W^2 + K \cdot W)$	Dense graphs where a few "spillable" webs would unjam things if removed.	7 / 7
T2.3	**Splitting, K**	$O(K \cdot W^2)$	Long-lived webs that interfere with many others over their lifetime.	4 / 4
T2.4	**Phantom · BCT-Color**	$O(W^2 + K \cdot (V + E))$	Hard cases. Petersen, Mycielski, Grötzsch — graphs that defeat naive greedy.	58 / 58

basic	edge cases	hourglass	stress	adversarial
6 / 6 PASS	22 / 22 PASS	10 / 10 PASS	12 / 12 PASS	25 / 25 PASS

One scaffold, four hooks. *All algorithms call* `basicColoring` *at their core. Spilling and splitting change what happens when it stalls; Phantom changes how nodes are ordered for selection — block by block.*

T1.3 DOCUMENTATION & COMPLEXITY SUMMARY

Doxygen everywhere, complexity proven per routine

Every public method carries `@brief`, `@param`, `@return`, and a complexity annotation. Generated HTML + LaTeX live under `docs/`.

ROUTINE	COMPLEXITY	NOTES
RangesParser::parse	$O(L)$	L = file char count.
WebBuilder::build	$O(R \cdot \alpha(R))$	Union-find over line sets.
InterferenceGraph::build	$O(W^2 \cdot \bar{L})$	Pairwise overlap check.
basicColoring	$O(W^2)$	Chaitin simplify + select.
selectSpillCandidate	$O(W)$	One pass over webs.
dsaturColoring	$O(W^2)$	Argmax re-scan per step.
findBiconnectedComponents	$O(V + E)$	Tarjan DFS.
colorBlock (exact)	$O(N^b)$	b = block size, small in practice.
reduceColors (Kempe)	$O(K \cdot (V + E))$	BFS per swap attempt.
OutputWriter::write	$O(W \log W)$	Sort by register, then web id.

75 / 75
TESTS PASSED

5
TEST SUITES

~3.2k
LOC CORE

100%
DOXYGEN COVERAGE

ADVERSARIAL SET HIGHLIGHTS

Petersen · $\chi = 3$, classic 3-regular cubic. **Mycielski-4** · triangle-free with $\chi = 4$. **Grötzsch** · the smallest triangle-free 4-chromatic graph.

> *These are the graphs that defeat naïve greedy. BCT-Color clears them all with no spilling — by exploiting block structure plus exact backtracking inside each block.*

Thank you.

Questions?

ALGORITHMS
4 / 4
Basic · Spilling · Splitting · BCT-Color (Tarjan + DSATUR + Kempe).

TEST PASS RATE
75 / 75
basic · edge cases · hourglass · stress · adversarial (Petersen, Mycielski, Grötzsch).

CORE COMPLEXITY
O(W²)
Chaitin simplify + select. BCT decomposition adds O(V + E) on top.

GROUP
T09 · G01
Pedro Tomás Teixeira · Rafael Pinho e Silva.

END OF PRESENTATION